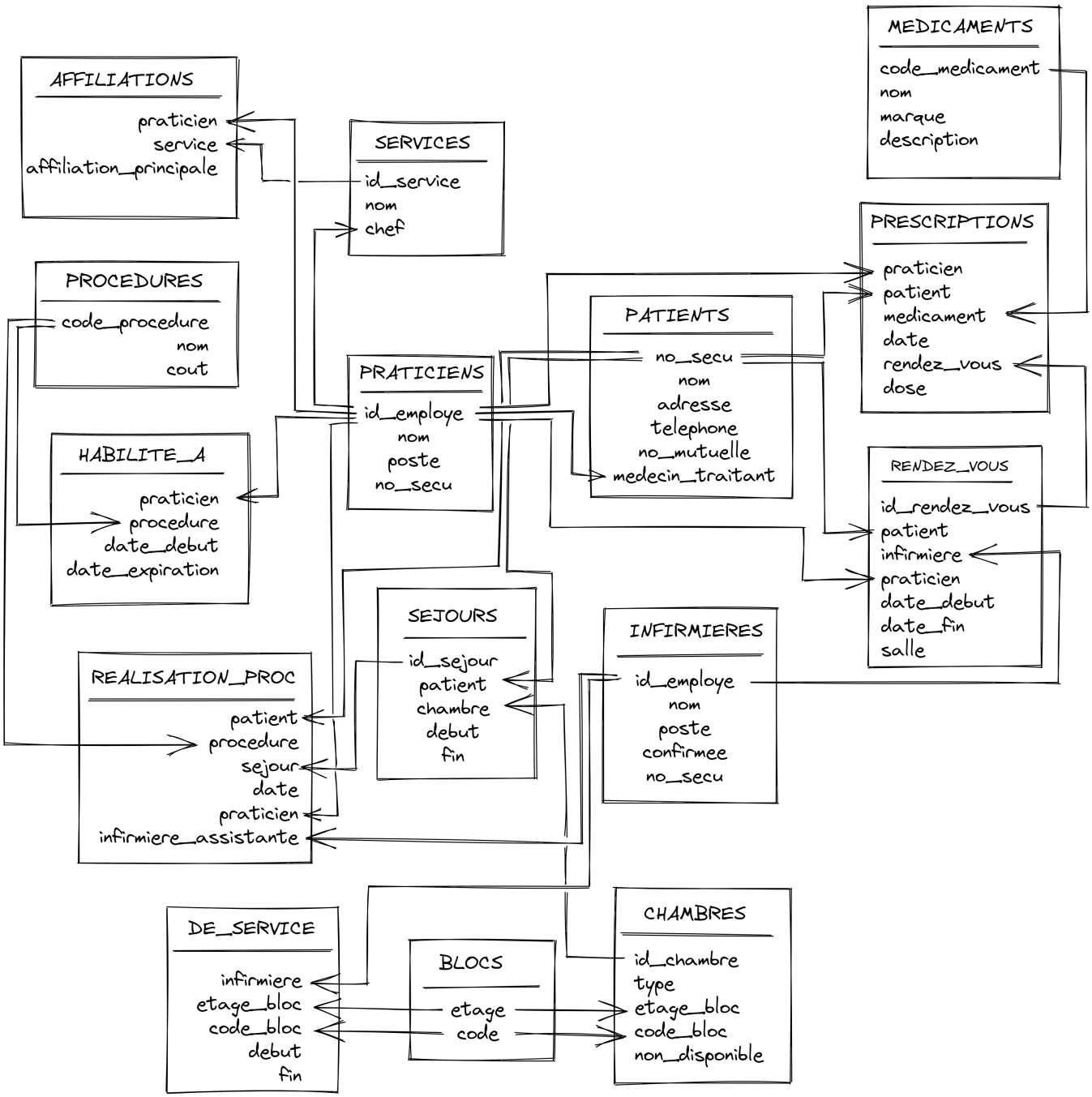
Enseignant :	Louis RAYNAL	**Établissement :**	ICES
Contact :	l-raynal@ices.fr	**Année :**	2023-2024
Public :	L3 Maths		
Ressources :	https://github.com/LouisRaynal/coursSQL		

Cours-TP 7 : Logique conditionnelle

Dans ce cours-TP nous utiliserons la base de données `hopital` rencontrée lors du TP 6. Son schéma relationnel est le suivant :



Dans certaines situations, vous voudrez que votre code effectue certaines actions en fonction des valeurs de colonnes ou expressions. Ce cours-TP présente comment écrire des requêtes qui se comportent différemment selon les données rencontrées lors de l'exécution. L'expression `CASE` nous permettra d'utiliser une telle logique conditionnelle.

1. Logique conditionnelle

La logique conditionnelle est simplement la capacité d'indiquer plusieurs chemins lors de l'exécution d'un code SQL. Dans une base de données il est courant d'utiliser des valeurs numériques (codées) au lieu de nomenclatures. Par exemple, dans la base de films `imdb`, la colonne `gender` de la table `realisators` codait le genre du réalisateur : 0 pour non renseigné/autre, 1 pour féminin, 2 pour masculin. Si vous êtes amenés à créer des rapports à partir de vos extractions SQL, vous voudrez traduire la signification de chaque code pour plus de lisibilité.

L'expression `CASE` de SQL va nous permettre d'effectuer ce genre d'actions de traduction, ainsi que d'autres plus complexes.

2. L'expression `CASE`

L'expression `CASE` va nous permettre d'écrire des instructions du type *if … then … else …* que l'on trouve dans les autres langages informatiques. `CASE` pourra s'utiliser dans des requêtes de type `SELECT`, `INSERT`, `UPDATE` ou `DELETE`. Dans ce cours nous nous concentrons sur les requêtes de sélection.

Il existe deux syntaxes pour `CASE` : l'**expression `CASE` élaborée** et l'**expression `CASE` simple**.

2.1. Expression `CASE` élaborée

Voici la première syntaxe de `CASE` que vous pouvez utiliser

Code 1 – Syntaxe d'expression `CASE` élaborée

```
CASE
    WHEN cond1 THEN exp1
    WHEN cond2 THEN exp2
    ...
    WHEN condN THEN expN
    ELSE expFin
END
```

Cette syntaxe requière la définition de conditions `cond1`, …, `condN` et d'expressions `exp1`, …, `expN`, `expFin`. Lorsqu'une condition dans une clause `WHEN` est vérifiée, l'expression correspondante est évaluée. De plus il est possible d'utiliser la clause `ELSE` pour définir l'expression qui sera évaluée si aucune des conditions `cond1`, …, `condN` n'est vérifiée. Cette clause `ELSE` est optionnelle.

Pratique

La requête ci-dessous fait la traduction du codage utilisé pour la colonne `confirmee` de la table `INFIRMIERES`.

Code 2 – Exemple d'expression `CASE` élaborée

```
SELECT
    nom,
    confirmee,
```

```
    CASE
        WHEN confirmee = 1 THEN 'Confirmé'
        WHEN confirmee = 0 THEN 'Non confirmé'
        ELSE 'Autre'
    END AS 'Statut infirmier'
FROM INFIRMIERES
;
```

Notez qu'il n'est pas nécessaire d'inclure la ligne `ELSE 'Autre'`, mais je vous le recommande tout de même. En effet, dans l'état présent de la base il n'y a pas d'autres modalités pour la colonne `confirmee`, mais mieux vaut prendre en compte toutes possibilités d'évolution lorsque vous rédigez des requêtes.

Attention

Lorsque vous exécutez une requête avec un `CASE`, pour chaque ligne de la table, les conditions sont parcourues de **haut en bas**. La première expression qui est vérifiée va retourner l'expression correspondante, et **les conditions suivantes seront ignorées**. De plus, pour une ligne donnée, si aucune condition n'est vérifiée et qu'il n'y a pas de clause `ELSE`, alors une valeur `NULL` est retournée.

Pratique

Afin d'illustrer le point d'attention ci-dessus, je vous invite à exécuter les deux requêtes ci-dessous :

Code 3 – Exemple de `CASE` avec condition toujours vraie

```
SELECT
    nom,
    confirmee,
    CASE
        WHEN 1 = 1 THEN 'ConditionTjrVraie'
        WHEN confirmee = 1 THEN 'Confirmé'
        WHEN confirmee = 0 THEN 'Non confirmé'
        ELSE 'Autre'
    END AS 'Statut infirmier'
FROM INFIRMIERES
;
```

Code 4 – Exemple de `CASE` sans clause `ELSE`

```
SELECT
    nom,
    confirmee,
    CASE
        WHEN confirmee = 1 THEN 'Confirmé'
    END AS "Statut infirmier"
FROM INFIRMIERES
;
```

Une expression `CASE` peut retourner n'importe quel type de données, par forcément une chaîne de caractères. Vous pouvez même retourner le résultat d'une sous-requête.

Pratique

La requête ci-dessous ajoute une colonne qui, pour chaque ligne de rendez-vous, retourne le nombre de consultations effectuées par l'infirmière concernée.

Code 5 – Exemple de `CASE` qui retounre le résultat d'une sous-requête

```
SELECT
    rdv.*,
    CASE
        WHEN rdv.infirmiere IS NOT NULL
            THEN (SELECT COUNT(rdv_bis.id_rendez_vous)
                    FROM RENDEZ_VOUS AS rdv_bis
                    WHERE rdv_bis.infirmiere = rdv.infirmiere )
        ELSE NULL
    END AS "Nombre de rendez-vous infirmière"
FROM RENDEZ_VOUS AS rdv
;
```

Cette requête utilise une sous-requête corrélée.

2.2. Expression `CASE` simple

L'expression `CASE` simple est assez similaire au `CASE` élaboré, mais est un peu moins flexible. Voici sa syntaxe :

Code 6 – Syntaxe d'expression `CASE` simple

```
CASE val0
    WHEN val1 THEN exp1
    WHEN val2 THEN exp2
    ...
    WHEN valN THEN expN
    ELSE expFinal
END
```

Pour cette syntaxe, des valeurs `val1`, . . ., `valN` sont comparées à une valeur `val0`. Lorsqu'une première égalité entre les valeurs est vérifiée, l'expression correspondante est retournée.

Pratique

Ci-dessous est l'équivalent de la requête de traduction de la colonne `confirmee` mais en utilisant une expression `CASE` simple.

Code 7 – Exemple d'expression `CASE` simple

```
SELECT
    nom,
    confirmee,
    CASE confirmee
        WHEN 1 THEN 'Confirmé'
        WHEN 0 THEN 'Non confirmé'
        ELSE 'Autre'
    END AS "Statut infirmier"
FROM INFIRMIERES
;
```

Une expression CASE simple est donc équivalente à sa version élaborée lorsqu'elle n'utilise que des conditions d'égalité de valeurs. La version élaborée permet cependant une large diversité de conditions, et plusieurs conditions peuvent même être utilisées dans une clause WHEN.

3. Exemples d'utilisations de CASE

Pour finir, nous allons voir des exemples d'utilisation de l'expression CASE pour différents usages.

3.1. Gestion des exceptions

Lorsque vous voulez obtenir des colonnes calculées à partir de divisions, vous devez toujours prendre en compte les divisions possibles par zéro. Certaines variantes de SQL gèrent cela en renvoyant des erreurs lors du calcul de la colonne, d'autres comme SQLite renverront une valeur NULL.

Essayez par exemple

```
SELECT 5 / 0 ;
```

Pour éviter des erreurs dues à la présence de valeurs NULL, il vaut mieux sécuriser vos calculs de division en utilisant un CASE au dénominateur.

Pratique

Imaginons que pour chaque rendez-vous, vous voulez calculer le ratio :
(Nombre total de rendez-vous du praticien / Nombre total de rendez-vous de l'infirmière)
Ça n'a pas trop de sens, c'est plus pour l'exemple.

Pour chaque ligne de rendez-vous, vous pouvez calculer les colonnes avec le nombre de rendez-vous du médecin et de l'infirmière grâce à des sous-requêtes :

Code 8 – Exemple de CASE pour gestion des divisions par zéros (partie 1)

```
SELECT
    rdv.*,
    ( SELECT COUNT(rdv_bis.id_rendez_vous)
      FROM RENDEZ_VOUS AS rdv_bis
      WHERE rdv_bis.praticien = rdv.praticien ) AS ExpMedecin,
    ( SELECT COUNT(rdv_bis.id_rendez_vous)
      FROM RENDEZ_VOUS AS rdv_bis
      WHERE rdv_bis.infirmiere = rdv.infirmiere ) AS ExpInfirmiere
FROM RENDEZ_VOUS AS rdv
;
```

Nous voulons maintenant faire la division à partir du résultat de cette requête. Cependant, lorsqu'il n'y a pas d'infirmière pour le rendez-vous nous voulons afficher uniquement la valeur pour le médecin, c'est-à-dire utiliser 1 pour le dénominateur de la division.

Code 9 – Exemple de CASE pour gestion des divisions par zéros

```
SELECT
    res.*,
    1.0 * res.ExpMedecin / CASE
                              WHEN res.ExpInfirmiere = 0 THEN 1
```

```
                                ELSE res.ExpInfirmiere
                            END
FROM
(
    SELECT
        rdv.*,
        ( SELECT COUNT(rdv_bis.id_rendez_vous)
          FROM RENDEZ_VOUS AS rdv_bis
          WHERE rdv_bis.praticien = rdv.praticien ) AS ExpMedecin,
        ( SELECT COUNT(rdv_bis.id_rendez_vous)
          FROM RENDEZ_VOUS AS rdv_bis
          WHERE rdv_bis.infirmiere = rdv.infirmiere ) AS ExpInfirmiere
    FROM RENDEZ_VOUS AS rdv
) res
;
```

Astuce

> Dans la requête précédente, nous avons multiplié le ratio par `1.0`, afin d'obtenir un résultat décimal. En effet, comme le numérateur et dénominateur sont tous les deux de type `INTEGER`, le résultat de la division sera de ce même type, d'où la multiplication par `1.0` pour forcer l'obtention d'un résultat de type `DECIMAL`.

3.2. Traduction de valeurs `NULL`

Bien que `NULL` soit la bonne valeur de colonne à stocker lorsque la valeur est inconnue, il n'est pas toujours approprié de retourner la valeur `NULL` à l'affichage ou dans une expression. Par exemple, vous voudrez plutôt afficher le mot "inconnu" dans un rapport, au lieu d'un champ vide. Vous pouvez utiliser l'expression `CASE` pour substituer ce mot si la valeur de colonne est `NULL`.

Pratique

Par exemple, en récupérant le nom des infirmières pour chaque ligne de rendez-vous, on souhaite plutôt afficher "Pas d'infirmière", de la manière suivante :

Code 10 – Exemple de `CASE` pour traduction de valeurs `NULL`

```
SELECT rdv.*,
    CASE
        WHEN infi.nom IS NULL THEN 'Pas d''infirmière'
        ELSE infi.nom
    END AS nomInfirmière
FROM RENDEZ_VOUS AS rdv
LEFT JOIN INFIRMIERES AS infi
    ON infi.id_employe = rdv.infirmiere
;
```

Étant donné que la plupart des opérations avec des valeurs `NULL` donnent un résultat `NULL`, il est courant d'utiliser un `CASE` pour gérer ces situations afin d'obtenir un résultat non-`NULL`.

3.3. Contrôle de l'existence

Pour finir, il s'agit maintenant de l'usage le plus pratique de `CASE` : utiliser `CASE` avec l'opérateur `EXISTS` dans le but de créer des colonnes personnalisées basées sur l'existence (des variables indicatrices par exemple).

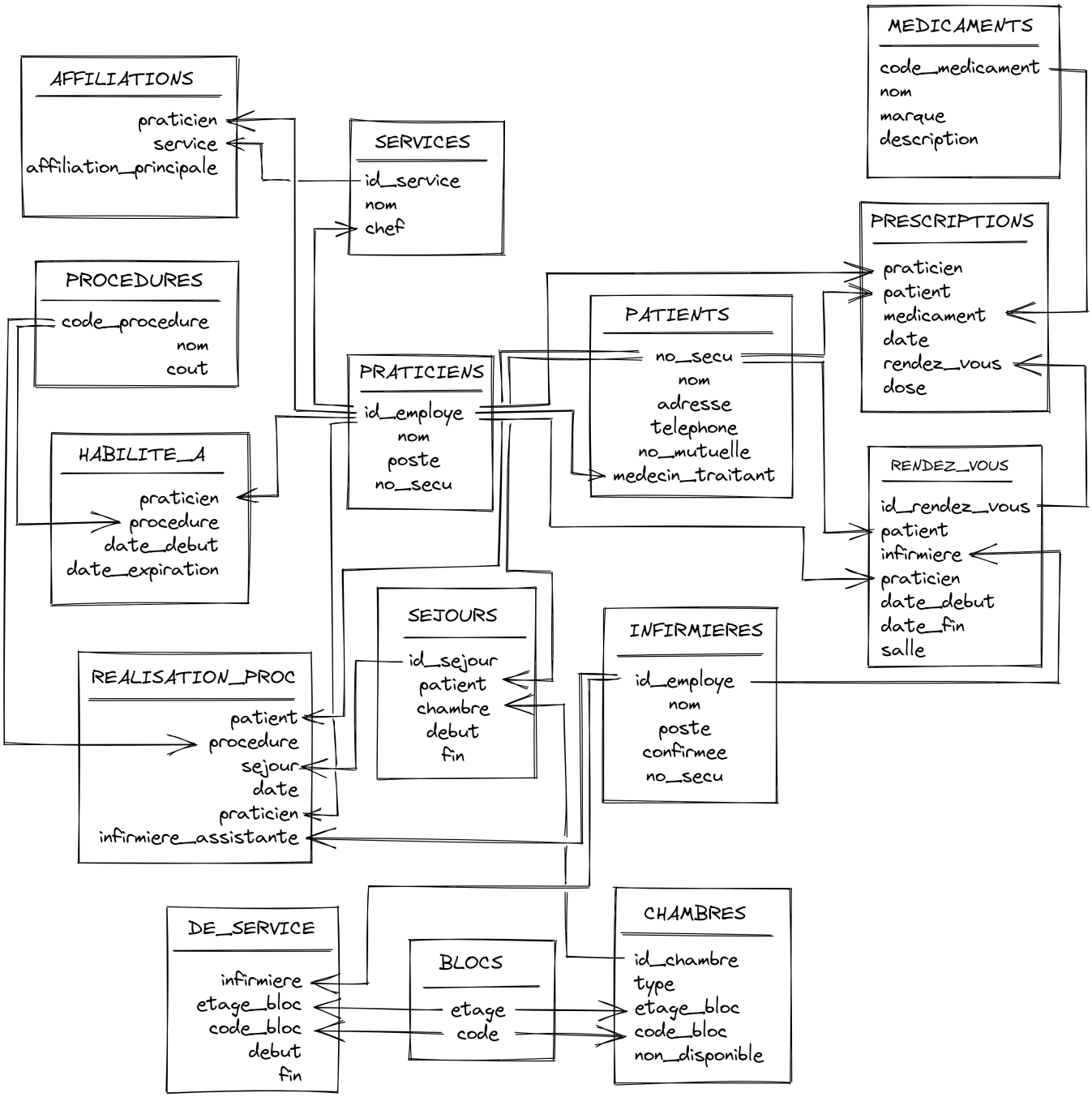
Pratique

Vous voulez ajouter une colonne intitulée "Prescription" dans la table `RENDEZ_VOUS`, qui indique si oui ou non un rendez-vous donné est associé à la prescription d'un médicament. Cela peut être fait très facilement en combinant `CASE` et `EXISTS`, de la manière suivante :

```sql
SELECT
    *,
    CASE
        WHEN EXISTS ( SELECT 1
                      FROM PRESCRIPTIONS AS presc
                      WHERE presc.rendez_vous = rdv.id_rendez_vous ) THEN 'OUI'
        ELSE 'NON'
    END AS Prescription
FROM RENDEZ_VOUS AS rdv
;
```

4. Exercices

Pour ces exercices vous utiliserez la base de données `hopital` pour laquelle le schéma relationnel est rappelé ci-dessous.



1. Listez le nom de praticiens, leur poste et service d'affiliation. En utilisant un `CASE` **simple**, créez une colonne "Type d'affiliation" qui affiche "Principale" lorsque l'affiliation est principale (codée 1), et "Secondaire" sinon. Créez ensuite une colonne "Chirurgien" qui prend pour valeur "OUI" si le nom du poste contient la chaîne de caractères 'surg' ou 'Surg', "NON" sinon.
2. Listez le nom et poste des praticiens, ainsi que le nom des procédures pour lesquelles ils ont été habilités. Créez une colonne "Spécialiste" qui indique "NON" si le praticien n'a aucune habilitation, "OUI" sinon.
3. Listez pour chaque salle, le nombre de rendez-vous ayant eu lieu dans cette salle en présence d'une infirmière. Créez ensuite une colonne "Assistance Rdv" qui prend pour valeur, pour une salle donnée, les valeurs "Importante" si plus de 2 rendez-vous ont été effectués en présence d'une infirmière dans la salle, "Moyenne" pour entre 1 et 2 rendez-vous, "Aucune" sinon.
4. Listez le nom et poste de toutes les infirmières. En combinant `CASE` et `EXISTS`, créez une colonne "Assistante", qui a pour valeurs : "Proc." si l'infirmière a assisté un praticien uniquement lors d'une procédure médicale, "Rdv." si uniquement lors d'un rendez-vous, "Proc. et Rdv." si lors d'un rendez-vous et d'une procédure médicale, "Non" sinon.
5. Listez le nom de tous les praticiens et infirmières. Au moyen d'un `CASE` et de sous-requêtes autonomes, créez dans un second temps une colonne "Profession" qui indique "Praticien" s'il s'agit d'un praticien, et "Infirmière" s'il s'agit d'une infirmière.
6. Listez tous les patients, et créez quatre colonnes : une première qui indique si oui ou non le patient a déjà été opéré, une deuxième qui indique combien de fois, une troisième qui indique si le patient a déjà reçu une prescription, une dernière qui indique le nombre de prescriptions reçues.